

CDN_V1.0.0_2026_07_28_ciszunetwork

CISZU NETWORK - DOCUMENTACIÓN OFICIAL

Nombre: CDN_MIGRATION_PLAN

Versión: 1.0.0

Actualización: 2026-07-28

Identificador: CDN_V1.0.0_2026_07_28_ciszunetwork

PLAN DE MIGRACIÓN A CDN MULTIMEDIA / CDN MULTIMEDIA MIGRATION PLAN

[ESPAÑOL]

1. PROBLEMA ACTUAL

El repositorio de Ciszu Network contiene (o referenciaba) assets multimedia pesados:

- GIFs animados
- Videos (MP4, WEBM)
- Imágenes grandes (PNG, JPG de banners, fondos)
- Archivos de música (MP3, OGG, WAV)
- Instaladores EXE (MuzicMania)

Aunque .gitignore excluye la mayoría (.gif, .mp4, *.mp3, etc.),

el contenido referenciado en el código usa rutas locales (public/, content/),

lo que impide:

- Servir assets optimizadamente (sin CDN, sin caché global)
- Mantener el repo liviano si se añadieran nuevos assets
- Sincronizar assets entre versiones (web, desktop, mobile)
- Escalar la distribución de contenido

2. SOLUCIÓN: SUPA BASE STORAGE COMO CDN

Supabase Storage (bucket ciszu-assets) actuará como CDN multimedia.

Ventajas:

- Sin costo adicional (incluido en el plan de Supabase)
- URLs públicas accesibles desde cualquier proyecto
- Integración directa con @ciszunetwork/cdn (resolver de assets ya implementado)
- Cache automático en el edge de Supabase
- Sin límite de ancho de banda significativo para nuestro volumen
- La librería @ciszunetwork/cdn ya tiene assetResolver.resolve() preparado

3. INFRAESTRUCTURA EXISTENTE

packages/cdn/index.ts:

resuelveIcon(name, style, format) → resuelve iconos SVG

assetResolver.resolve(path) → resuelve cualquier asset (preparado para CDN)

scripts/upload-cdn.js:

Sube assets a Supabase Storage usando SUPABASE_SERVICE_ROLE_KEY

Comando: pnpm cdn:upload

scripts/copy-assets.js:

Fallback offline: copia SVGs críticos a public/ durante prebuild

4. INVENTARIO DE ASSETS POR PROYECTO

4.1 Ciszunetwork Page (apps/website)

- public/images/ — Banners, fondos, logos
- public/icons/ — Iconos (ya en shared/icons/svg/)
- public/logos/ — Logotipos

4.2 Ciszuko Antony Portfolio (apps/ciszukoantony/website)

- public/images/ — Imágenes de perfil, proyectos, galería
- public/logos/ — Logos personales

4.3 MuzicMania (apps/muzicmania)

- content/music/ — Álbum genesis_neon y subálbumes (pesado)
- content/logos/ — GIFs, imágenes, videos de logotipos
- content/arrowskins/ — 384 SVGs de flechas
- content/particleskins/ — Skins de partículas
- public/downloads/ — Instaladores EXE (MuzicMania v2.0.1)
- public/images/ — Imágenes varias
- public/fonts/ — Fuentes tipográficas

4.4 Ciszubot (apps/ciszubot)

- website/public/images/ — Imágenes del bot, banners
- website/public/icons/ — Iconos

4.5 Cizugamens (cizugamens/)

- Multimedia de comunidad (banners, logos, thumbnails) — pendiente de crear

5. PLAN DE MIGRACIÓN — 5 FASES

FASE 1: INVENTARIO Y PREPARACIÓN

- [] Catalogar todos los assets multimedia de cada proyecto
- [] Decidir qué va a CDN vs qué queda local (solo SVGs críticos y favicons locales)
- [] Actualizar scripts/upload-cdn.js si es necesario

FASE 2: SUBIDA A SUPA BASE

- ☐ Ejecutar `pnpm cdn:upload` para cada proyecto
- ☐ Verificar que los archivos aparecen en Supabase Storage (`ciszu-assets`)
- ☐ Documentar las URLs públicas resultantes

FASE 3: ACTUALIZACIÓN DE CÓDIGO

- ☐ Modificar cada webpage para usar `assetResolver.resolve()` en lugar de rutas locales
- ☐ Actualizar componentes que referencian imágenes, videos, audio
- ☐ Para MuzicMania: migrar referencias en el motor de juego, skins, música
- ☐ Para el website principal: actualizar banners, logos, favicons
- ☐ Para el portfolio: actualizar galería de proyectos
- ☐ Para CiszuBot: actualizar imágenes del website

FASE 4: VERIFICACIÓN Y TESTING

- ☐ Probar cada webpage en desarrollo local
- ☐ Verificar que assets se cargan desde CDN (no desde archivos locales)
- ☐ Verificar fallback offline (`copy-assets.js`) para SVGs críticos
- ☐ Probar en dispositivos y conexiones lentas

FASE 5: LIMPIEZA DEL REPOSITORIO

- ☐ Eliminar assets migrados del tracking de git (actualizar `.gitignore`)
- ☐ Eliminar archivos locales que ya no son necesarios (opcional, se pueden mantener como respaldo)
- ☐ Verificar reducción de peso del repositorio
- ☐ Hacer `commit + push + deploy`

6. INTEGRACIÓN CON TURBOMONOREPO

La migración a CDN es el primer paso hacia la reestructuración a Turbomonorepo.

Una vez que los assets están en CDN:

- Los proyectos pesan menos y compilan más rápido
- El build cache de Turborepo es más efectivo
- Podemos centralizar configuraciones compartidas (TypeScript, Tailwind, ESLint)
- El pipeline de CI/CD se vuelve más liviano
- La estructura del monorepo puede simplificarse

Pasos hacia Turbomonorepo:

1. ☐ CDN Migration (assets fuera del repo) — EN CURSO
2. Crear `packages/config/` para configs compartidas (`tsconfig`, `tailwind`, `eslint`)
3. Crear `packages/ui/` para componentes compartidos
4. Unificar build pipeline con Turborepo remote caching

5. Centralizar esquemas de Supabase

7. CENTRALIZACIÓN DE SUPA BASE (Futuro)

Después de CDN:

- Unificar todos los esquemas de base de datos en un proyecto Supabase central
- Compartir migraciones entre proyectos
- Políticas RLS unificadas
- Un solo conjunto de credenciales para toda la red

8. BENEFICIOS ESPERADOS

- Repositorio Git más liviano (clonados más rápidos)
- Carga de assets más rápida (edge CDN de Supabase)
- Sincronización automática entre versiones web/desktop/mobile
- Menos dependencia de archivos locales
- Mejor experiencia de desarrollo (no esperar descargas de assets pesados)
- Preparación para escalar la red

9. RIESGOS Y MITIGACIONES

Riesgo: Supabase Storage caído

Mitigación: Fallback offline (copy-assets.js) mantiene SVGs críticos locales

Riesgo: Cambio de URLs de CDN en el futuro

Mitigación: `assetResolver.resolve()` abstrae la URL — solo cambiar en un lugar

Riesgo: Ancho de banda de Supabase excedido

Mitigación: El bucket `ciszu-assets` tiene límite de 50 MB, monitorear uso

[ENGLISH]

1. CURRENT PROBLEM

The Ciszu Network repository contains (or referenced) heavy multimedia assets:

- Animated GIFs
- Videos (MP4, WEBM)
- Large images (PNG, JPG banners, backgrounds)
- Music files (MP3, OGG, WAV)
- EXE installers (MuzicMania)

Although `.gitignore` excludes most (`.gif`, `.mp4`, `*.mp3`, etc.),
the code references use local paths (`public/`, `content/`),

which prevents:

- Optimized asset serving (no CDN, no global cache)
- Keeping the repo lightweight if new assets are added
- Syncing assets between versions (web, desktop, mobile)
- Scaling content distribution

2. SOLUTION: SUPABASE STORAGE AS CDN

Supabase Storage (ciszu-assets bucket) will act as multimedia CDN.

Advantages:

- No additional cost (included in Supabase plan)
- Public URLs accessible from any project
- Direct integration with @ciszunetwork/cdn (asset resolver already implemented)
- Automatic edge caching by Supabase
- No significant bandwidth limit for our volume
- The @ciszunetwork/cdn package already has `assetResolver.resolve()` ready

3. EXISTING INFRASTRUCTURE

`packages/cdn/index.ts`:

`resolveIcon(name, style, format) → resolves SVG icons`

`assetResolver.resolve(path) → resolves any asset (CDN-ready)`

`scripts/upload-cdn.js`:

Uploads assets to Supabase Storage using `SUPABASE_SERVICE_ROLE_KEY`

Command: `pnpm cdn:upload`

`scripts/copy-assets.js`:

Offline fallback: copies critical SVGs to `public/` during prebuild

4. ASSET INVENTORY BY PROJECT

4.1 Ciszunetwork Page (apps/website)

- `public/images/` — Banners, backgrounds, logos
- `public/icons/` — Icons (already in `shared/icons/svg/`)
- `public/logos/` — Logotypes

4.2 Ciszuko Antony Portfolio (apps/ciszukoantony/website)

- `public/images/` — Profile images, projects, gallery
- `public/logos/` — Personal logos

4.3 MuzicMania (apps/muzicmania)

- `content/music/` — `genesis_neon` album and sub-albums (heavy)
- `content/logos/` — GIFs, images, videos of logotypes

- content/arrowskins/ — 384 arrow SVGs
- content/particleskins/ — Particle skins
- public/downloads/ — EXE installers (MuzicMania v2.0.1)
- public/images/ — Various images
- public/fonts/ — Typography fonts

4.4 Ciszubot (apps/ciszubot)

- website/public/images/ — Bot images, banners
- website/public/icons/ — Icons

4.5 Cizugamens (cizugamens/)

- Community multimedia (banners, logos, thumbnails) — pending creation

5. MIGRATION PLAN — 5 PHASES

PHASE 1: INVENTORY AND PREPARATION

PHASE 2: UPLOAD TO SUPABASE

PHASE 3: CODE UPDATE

PHASE 4: VERIFICATION AND TESTING

PHASE 5: REPOSITORY CLEANUP

6. TURBOMONOREPO INTEGRATION

CDN migration is the first step towards Turbomonorepo restructuring.

Once assets are on CDN:

- Projects are lighter and compile faster
- Turborepo build cache is more effective
- Shared configurations can be centralized
- CI/CD pipeline becomes lighter
- Monorepo structure can be simplified

Steps towards Turbomonorepo:

1. ☐ CDN Migration (assets out of repo) — IN PROGRESS
2. Create packages/config/ for shared configs (tsconfig, tailwind, eslint)
3. Create packages/ui/ for shared components
4. Unify build pipeline with Turborepo remote caching
5. Centralize Supabase schemas

7. SUPA BASE CENTRALIZATION (Future)

After CDN:

- Unify all database schemas in a central Supabase project

- Share migrations between projects
- Unified RLS policies
- Single set of credentials for the entire network

8. EXPECTED BENEFITS

- Lighter Git repository (faster clones)
- Faster asset loading (Supabase edge CDN)
- Automatic sync between web/desktop/mobile versions
- Less dependency on local files
- Better development experience (no heavy asset downloads)
- Preparation for scaling the network